```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sklearn

print("All libraries are ready!")
```

```
All libraries are ready!
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/autos/imports-85.data"

columns = [
    "symboling", "normalized_losses", "make", "fuel_type",
    "aspiration", "num_doors", "body_style", "drive_wheels",
    "engine_location", "wheel_base", "length", "width",
    "height", "curb_weight", "engine_type", "num_cylinders",
    "engine_size", "fuel_system", "bore", "stroke",
    "compression_ratio", "horsepower", "peak_rpm",
    "city_mpg", "highway_mpg", "price"
]

df = pd.read_csv(url, names=columns, na_values="?")

print("Dataset loaded successfully!")
print(df.head())
print("Dataset shape:", df.shape)
```

```
Dataset loaded successfully!
   symboling  normalized_losses         make fuel_type aspiration num_doors  \
0          3                NaN  alfa-romero       gas        std       two
1          3                NaN  alfa-romero       gas        std       two
2          1                NaN  alfa-romero       gas        std       two
3          2              164.0         audi       gas        std      four
4          2              164.0         audi       gas        std      four

    body_style drive_wheels engine_location  wheel_base  ...  engine_size  \
0  convertible          rwd           front        88.6  ...          130
1  convertible          rwd           front        88.6  ...          130
2    hatchback          rwd           front        94.5  ...          152
3        sedan          fwd           front        99.8  ...          109
4        sedan          4wd           front        99.4  ...          136

   fuel_system  bore  stroke compression_ratio horsepower  peak_rpm city_mpg  \
0         mpfi  3.47    2.68               9.0      111.0    5000.0       21
1         mpfi  3.47    2.68               9.0      111.0    5000.0       21
2         mpfi  2.68    3.47               9.0      154.0    5000.0       19
3         mpfi  3.19    3.40              10.0      102.0    5500.0       24
4         mpfi  3.19    3.40               8.0      115.0    5500.0       18

   highway_mpg    price
0           27  13495.0
1           27  16500.0
2           26  16500.0
3           30  13950.0
4           22  17450.0

[5 rows x 26 columns]
Dataset shape: (205, 26)
```

```
df["price"] = pd.to_numeric(df["price"], errors="coerce")
df = df.dropna(subset=["price"]).copy()

X = df.drop(columns=["price"])
y = df["price"]

print("Missing target prices removed.")
print("Input features:", X.shape)
print("Target values:", y.shape)
```

```
Missing target prices removed.
Input features: (201, 25)
Target values: (201,)
```

```
numeric_features = X.select_dtypes(include=np.number).columns.tolist()
categorical_features = X.select_dtypes(exclude=np.number).columns.tolist()

preprocessor = ColumnTransformer([
    ("num", SimpleImputer(strategy="median"), numeric_features),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
    ]), categorical_features)
])

model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model.fit(X_train, y_train)

print("Car Price Prediction model trained successfully!")
```

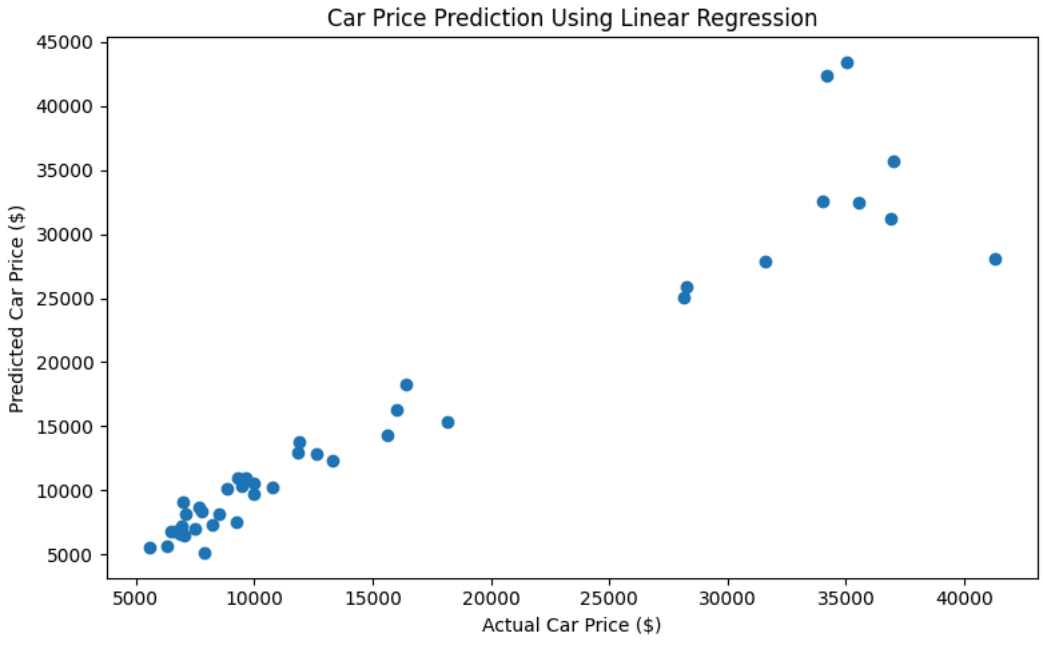
```
Car Price Prediction model trained successfully!
```

```
y_pred = model.predict(X_test)

print("Mean Absolute Error:", round(mean_absolute_error(y_test, y_pred), 2))
print("Mean Squared Error:", round(mean_squared_error(y_test, y_pred), 2))
print("RMSE:", round(np.sqrt(mean_squared_error(y_test, y_pred)), 2))
print("R2 Score:", round(r2_score(y_test, y_pred), 3))
```

```
Mean Absolute Error: 1972.15
Mean Squared Error: 10576265.64
RMSE: 3252.12
R2 Score: 0.914
```

```
plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Car Price ($)")
plt.ylabel("Predicted Car Price ($)")
plt.title("Car Price Prediction Using Linear Regression")
plt.tight_layout()
plt.show()
```




```
new_car = X.iloc[[0]]
predicted_price = model.predict(new_car)[0]

print("Sample car details:")
```

```
print(new_car)
print("\nPredicted car price: $", round(predicted_price, 2))
```

```
Sample car details:
   symboling  normalized_losses         make fuel_type aspiration num_doors  \
0          3                NaN  alfa-romero       gas        std       two

    body_style drive_wheels engine_location  wheel_base  ...  num_cylinders  \
0  convertible          rwd           front        88.6  ...           four

   engine_size  fuel_system  bore stroke compression_ratio  horsepower  \
0          130         mpfi  3.47   2.68               9.0       111.0

  peak_rpm  city_mpg  highway_mpg
0   5000.0        21           27

[1 rows x 25 columns]

Predicted car price: $ 15301.34
```

Start coding or generate with AI.